

PRAKTIKUM SISTEM OPERASI

MODUL 6

Proses Sekuensial dan Konkuren



Oleh:

Martryatus Sofia

2311104003

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

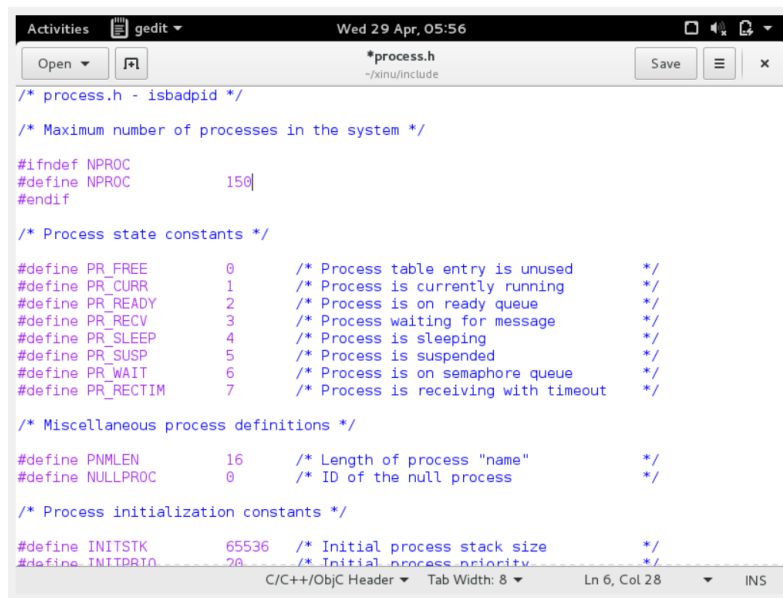
2026

JURNAL

1. Soal 1

[10 Poin] Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah `$cat /proc/sys/kernel/pid_max`.

Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!



```
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifdef NPROC
#define NPROC      150
#endif

/* Process state constants */

#define PR_FREE      0      /* Process table entry is unused */
#define PR_CURR      1      /* Process is currently running */
#define PR_READY     2      /* Process is on ready queue */
#define PR_RECV      3      /* Process waiting for message */
#define PR_SLEEP     4      /* Process is sleeping */
#define PR_SUSP      5      /* Process is suspended */
#define PR_WAIT      6      /* Process is on semaphore queue */
#define PR_RECTIM    7      /* Process is receiving with timeout */

/* Miscellaneous process definitions */

#define PNMLEN      16      /* Length of process "name" */
#define NULLPROC    0      /* ID of the null process */

/* Process initialization constants */

#define INITSTK      65536  /* Initial process stack size */
#define INITPRIO     20     /* Initial process priority */
```

2. Soal 2

[20 Poin] Jalankan kode `main_ex1`!

```
Activities Terminal Wed 29 Apr, 06:17 xinu@xinu-develop-end: ~/xinu/compile
```

```
File Edit View Search Terminal Help
```

```
Xinu for Vbox -- version #157   (xinu)   Wed 29 Apr 09:15:40 EDT 2026
```

```
Found Intel 82545EM Ethernet NIC  
MAC address is 08:00:27:9e:f4:08  
4447448 bytes of free memory. Free list:  
    [0x0015E320 to 0x0009FFF7]  
    [0x00100000 to 0x005F8FFF]  
116776 bytes of Xinu code.  
    [0x00100000 to 0x0011C827]  
148212 bytes of data.  
    [0x00120000 to 0x001442F3]
```

```
Obtained IP address 10.0.3.15   (0x0a00030f)
```

```
A  
A  
A  
A  
A  
A  
A  
A  
A
```

```
CTRL-A Z for help | 115200 8N1 | N0R | Minicom 2.7 | VT102 | Offline | ttyS0
```

3. Soal 3

[20 Poin] Jalankan kode main_ex2!

[illegible]

4. Soal 4

[20 Poin] Jalankan kode main_ex3!

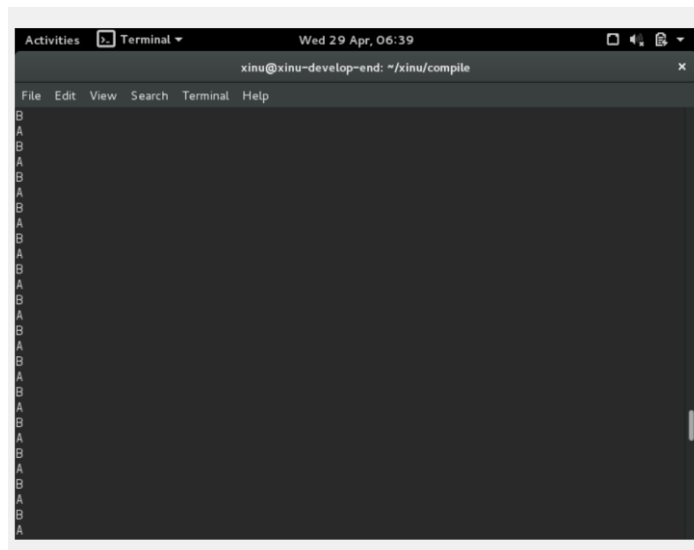
```

#include <xinu.h>
void sndChar(char);

process main(void)
{
    resume(create(sndChar, 1024, 20, "send A", 1, 'A'));
    resume(create(sndChar, 1024, 20, "send B", 1, 'B'));
    return OK;
}

void sndChar(char c)
{
    while(1){
        printf("%c \n", c);
        sleepms(1000);
    }
}

```



5. Soal 5

[30 Poin] Buatlah 2 proses produser dan konsumen. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```
int32 n = 0; //global variabel

void produser(void){
    int32 i;
    for (i=1; i<=1000; i++){
        n++;
    }
}

void konsumen(void){
    int32 i;
    for (i=1; i<=1000; i++){
        printf("Nilai dari n adalah %d\n",n);
    }
}
```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal).

Jelaskan mengapa hasilnya seperti itu!

```
#include <xinu.h>

int32 n = 0;

process produser(void){
    int32 i ;

    for (i = 1; i <= 1000; i++){
        n++;
    }

    return OK;
}

process konsumer(void){
    int32 i;

    for (i = 1; i <=1000; i++){
        kprintf("Nilai dari n adalah %d\n", n);
    }

    return OK;
}
```

[illegible]

Program menghasilkan output yang tidak selalu sesuai intuisi karena proses produser dan konsumen berjalan secara konkuren. Variabel n adalah variabel global yang digunakan bersama oleh kedua proses. Proses produser menaikkan nilai n dari 0 sampai 1000, sedangkan proses konsumen menampilkan nilai n sebanyak 1000 kali. Karena tidak ada sinkronisasi seperti semaphore, proses konsumen bisa membaca nilai n saat produser belum selesai. Akibatnya nilai yang ditampilkan bisa berbeda-beda, misalnya masih 0, nilai tengah, atau sudah 1000. Kondisi ini disebut race condition.